

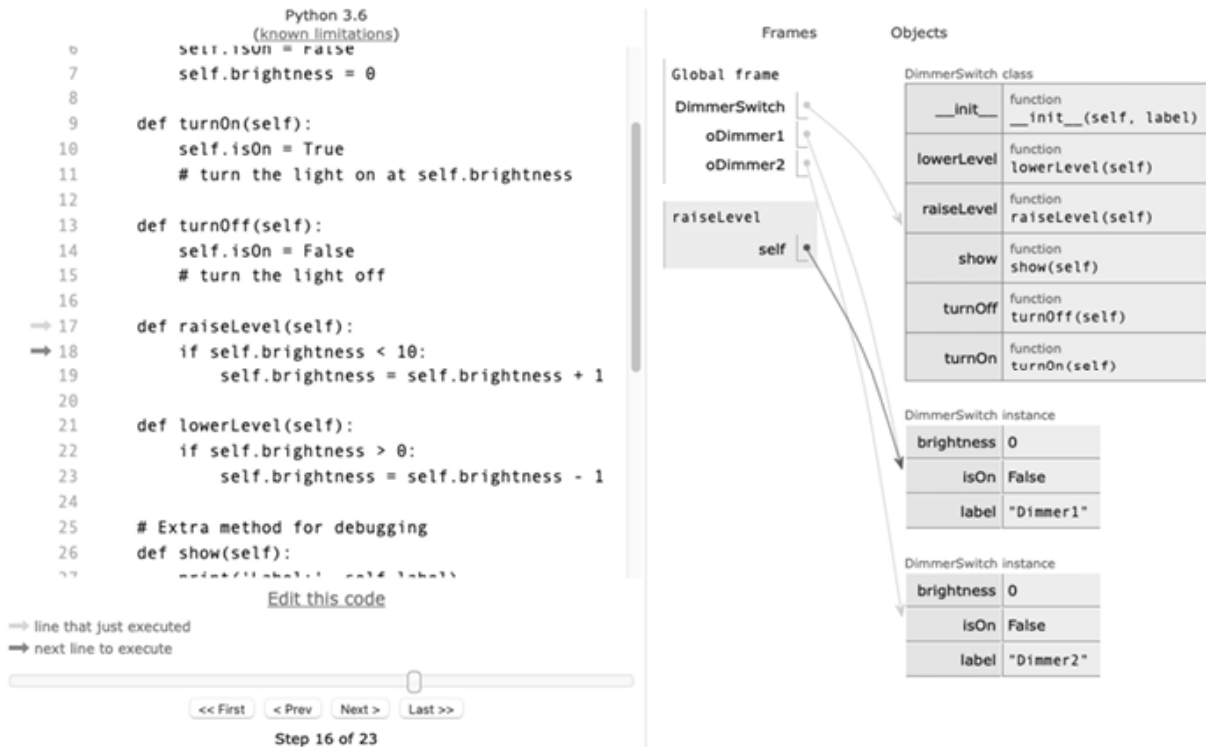


Figure 3-5: The program in [Listing 3-3](#) stopped in call to `oDimmer1.raiseLevel()`

Notice that `self` and `oDimmer1` refer to the same object. When the method executes and uses any `self.<instanceVariable>`, it will use `oDimmer1`'s instance variables. Therefore, when this method runs, `self.brightness` refers to the `brightness` instance variable in `oDimmer1`.

If we continue to execute the test code in [Listing 3-3](#), we get to the second call to `raiseLevel()` with `oDimmer2`. In [Figure 3-6](#), I've stopped execution inside this method call.

Notice that this time, `self` refers to the same object as `oDimmer2`. Now, `self.brightness` refers to the `brightness` instance variable of `oDimmer2`.

No matter what object we use or which method we call, the value of the object is assigned to the variable `self` in the called method. You should think of `self` as meaning the current object—the object that the method was called with. Whenever a method executes, it uses the set of instance variables for the object specified in the call.